

APAPO

Universal Hypergrid Platform

Complete System Design Document

Document Type	Complete Design Document — Reference Architecture
Version	1.0
Date	March 2026
Status	Authoritative Reference
Systems Covered	Hypergrid · Kogi · Ume · Qala
Classification	Confidential — Internal Use Only
Primary Languages	Rust · Go · TypeScript · SQL

Table of Contents

Part I — Executive Summary & Vision

1. Introduction

1.1 What is Apapo?

Apapo is the canonical name for the complete Hypergrid-based universal platform ecosystem — the integrated whole formed by the Hypergrid N-Dimensional Distributed Spreadsheet System (NDSS) substrate and all domain operating systems that run on top of it: Kogi (Independent Worker OS), Ume (Business OS), and Qala (Solution Factory OS). Where Hypergrid is the engine, Apapo is the vehicle — the living, federated, AI-augmented, multi-domain platform that transforms how individuals, organizations, and solution factories manage their data, operations, and intelligence.

The name Apapo encodes the platform's essential character: it is a convergence point. It is the place where independent workers (Kogi), organizations (Ume), and solution producers (Qala) share a single living data substrate, connected by a universal graph, governed by a unified permission model, and made intelligent by a shared AI engine. Every entity in Apapo — from a solo freelancer's portfolio item to a multinational organization's legal entity to a regulated pharmaceutical solution — is a HyperRow in a domain-specific Hypercube in the same Universal Cell Store, versioned by the same EventLog, synchronized by the same CRDT protocol, and reasoned over by the same AI framework.

1.2 The Central Architectural Thesis

Every complex software system faces the same underlying challenge: how to store, version, relate, compute over, and distribute structured data about entities that exist in multiple contexts simultaneously, change over time, and are operated on collaboratively. Apapo's thesis is that a generalized N-dimensional distributed spreadsheet — Hypergrid — is the universal answer to this challenge. All domain systems in the Apapo ecosystem are lenses over this shared substrate.

Traditional platform engineering solves this problem by designing bespoke schemas for each domain, each team, and each application — paying the full infrastructure cost of distributed consistency, versioning, graph modeling, permission systems, and event sourcing again and again, each time from scratch, in incompatible ways. This approach produces systems that cannot talk to each other, cannot share data without transformation pipelines, cannot maintain consistent audit trails across domains, and cannot be extended without engineering effort.

Apapo's approach is fundamentally different. The N-dimensional spreadsheet is a universal model. Every domain entity can be described as 'something with properties (columns), organized into groups (rows), that exists across multiple contexts simultaneously (extra dimensions).' Hypergrid makes that model distributed, collaborative, time-travelable,

graph-aware, intelligent, and infinitely extensible — and then provides it as the shared substrate from which any domain-specific system is built. Domain systems contribute domain logic, domain vocabulary, domain workflows, and domain UIs — but they inherit all data infrastructure from Hypergrid for free.

1.3 The Three Domain Operating Systems

System	Domain	Core Metaphor	Primary Entity	Scale Target
Kogi	Independent Worker OS	Portfolio as living workspace	Component (Item Container)	Individual to cooperative
Ume	Organization / Business OS	Organization as supervised kernel	OrgModule (42 domain areas)	Team to enterprise
Qala	Solution Factory OS	Factory producing governed solutions	SDE / Solution	Solo developer to global enterprise

Despite their wildly different domains, all three operating systems share identical underlying needs: entities with typed properties, lifecycle states, version history, graph relationships, CRDT-safe collaborative editing, immutable audit trails, AI-computed attributes, and time-travel queries. Hypergrid provides all of these at the substrate level so each domain system can focus entirely on its domain logic rather than reinventing data infrastructure.

1.4 Guiding Design Principles

- **Universal Abstraction** — Every platform entity across all three domain systems is a HyperRow. The spreadsheet metaphor generalizes to all domains without loss of expressiveness.
- **Distributed-First** — The system is designed for multi-node, multi-device, offline-capable operation. VectorClocks and CRDTs are not features added later; they are structural primitives.
- **Event-Sourced Truth** — Every mutation is an immutable EventLog entry. Nothing is ever deleted — only archived or superseded. Time-travel to any historical state is a first-class operation.
- **Identity Sovereignty** — Every entity owns its data, its identities, its visibility rules, and its connections. The permission model and VisibilityMask are substrate-level, not application-level.
- **AI-Augmented Intelligence** — The spreadsheet is not passive storage. It actively surfaces intelligence, computes signals, flags risks, and suggests actions. AI computations are first-class HyperCell attributes.
- **Open Extensibility** — The HypercubePlugin trait and domain pack system allow platform extensions without forking the substrate. New dimension types, attribute types, AI engines, and render modes are all first-class extension points.
- **Connected Economy** — Every entity is a node in a global federated graph. Value, data, and intelligence flow through HypergraphEdges across domain boundaries, tenants, and federation nodes.

Part II — Hypergrid Substrate

2. The Hyperspreadsheet as Universal Abstraction

2.1 Why a Spreadsheet?

The spreadsheet is the most widely understood data model in the world. Virtually every domain expert — a financial analyst, a project manager, a scientist, a supply chain planner — reaches for a spreadsheet when they need to organize structured data. The spreadsheet model captures three things that humans naturally think about data: entities have properties (rows × columns), entities exist in groups (multiple rows), and entities can be compared (visual layout).

The problem with the conventional spreadsheet is that it only models two dimensions — rows and columns — and it is not distributed, versioned, or AI-aware. Hypergrid generalizes the spreadsheet to N dimensions, making it distributed-first, CRDT-native, graph-connected, and AI-augmented. This generalization turns the most familiar data model in the world into a universal substrate for arbitrary domain systems.

2.2 Formal Definition of the N-Dimensional Model

Let a Hypercube H be defined by an ordered tuple of N dimension axes:

```
H = (D1, D2, ..., DN)

where:
  Di = DimensionAxis(id, name, type, key_set, ordering, cardinality)
  N   = the dimensionality of the Hypercube (N ≥ 2)

A HyperCell C is a data point addressed by a coordinate tuple:
  C = Cell(k1 ∈ D1.key_set, k2 ∈ D2.key_set, ..., kN ∈ DN.key_set)

Each HyperCell carries an attribute map:
  C.attributes = { a1: v1, a2: v2, ..., aN: vN }
  where ai ∈ AttributeKeyRegistry and vi is a TypedAttrValue

The Universal Cell Store (UCS) holds all HyperCells across all Hypercubes:
  UCS = { (grid_id, cube_id, dim_keys_tuple) → HyperCell }

A conventional 2D spreadsheet is the N=2 special case:
  D1 = RowAxis      (entity dimension)
  D2 = ColumnAxis   (attribute/property dimension)
  C.attributes = { "value": V, "type": T, "format": F }
```

2.3 Dimension Axis Model

Field	Type	Description
axis_id	UUID	Globally unique identifier for this axis within the grid

Field	Type	Description
name	String	Human-readable name: 'rows', 'columns', 'time', 'region', 'org_unit', etc.
dim_index	u8 (1–255)	Position in the ordered dimension tuple. $D_1=1$ (rows), $D_2=2$ (columns), D_3-D_n =custom
axis_type	AxisType enum	EntityAxis PropertyAxis TimeAxis GeoAxis CategoryAxis OrdinalAxis HierarchyAxis GraphAxis Custom
key_type	KeyType enum	Uuid String Integer Ordinal Timestamp GeoHash HierarchyPath Custom
key_set	KeySet	The set of valid keys: Enumerated (dense) or schema-constrained (sparse/infinite)
key_cardinality	KeyCardinality	Dense(max_keys) Sparse Infinite — affects storage encoding and index choice
ordering	AxisOrdering	Unordered Lexicographic Numeric Chronological Custom(CompareFn)
hierarchy	Option<HierarchyDef>	If HierarchyAxis: parent-child relationships between keys for drill-down
visibility	AxisVisibility	Public Tenant Private — governs which tenants/identities can see this axis
nullable	bool	If true: HyperCells need not exist for every key combination (sparse)
crdt_semantics	CrdtSemantics	LWW OR-Set Counter Lattice — CRDT behavior for mutations along this axis
index_strategy	IndexStrategy	BTree Hash GIN BRIN TimeSeries Spatial — storage index type

2.4 Built-in Axis Types

AxisType	Key Type	Ordering	Typical Use	Special Behaviors
EntityAxis	Uuid / String	Unordered	D_1 (rows) — primary entity dimension: users, projects, products	Full record model. GraphEdge::Hierarchy supported. CRDT: LWW per attribute
PropertyAxis	String	Lexicographic	D_2 (columns) — property/attribute dimension: field names, metric names	Column type registry. ComputedAttribute support. VisibilityMask per key
TimeAxis	Timestamp	Chronological	D_3 — temporal slicing: snapshots, time-series, event history	AS_OF time-travel queries. Automatic snapshots. BRIN index. Rollup by period

AxisType	Key Type	Ordering	Typical Use	Special Behaviors
GeoAxis	GeoHash / Coord	Spatial	D ₄ — geographic slicing: region, country, city, lat/lng bucket	Spatial index (PostGIS). Drill-down: continent → country → city
CategoryAxis	String (enum)	Ordinal	D ₃ –N — classification: product line, risk tier, department	Enum key validation. Hierarchy support. Cross-category pivot
HierarchyAxis	HierarchyPath	Tree (DFS)	D ₃ –N — org chart, product taxonomy, tag hierarchy	Parent-child key relationships. Rollup propagation. Drill-down/drill-up
TenantAxis	TenantId	Unordered	D ₃ — multi-tenancy across a single Hypercube	Row-level security: each key visible only to matching tenant
ScenarioAxis	String	Custom	D ₃ –N — what-if and planning: base, optimistic, pessimistic	Scenario comparison views. Variance across keys. Plan-vs-actual analysis
VersionAxis	SemVer / Integer	Numeric	D ₃ — schema/data versioning across cube schema migrations	Fork-and-merge version operations. Diff between version keys
Custom	Plugin-defined	Plugin-defined	Any domain-specific axis not covered above	HypercubePlugin provides key_type, validation, index, and rendering

2.5 Dimensionality Constraints

Constraint	Value	Rationale
Minimum dimensions (N)	2	2D is the degenerate case: a conventional spreadsheet. All Hypergrid ops valid at N=2
Maximum dimensions (N)	16 (hard limit)	Beyond 16 dims, query planning complexity exceeds practical utility. Recommended max: 6
Max keys per dense axis	10,000,000 (10M)	Dense axes are fully materialized. Beyond 10M keys, use SparseAxis with on-demand materialization
Max keys per sparse axis	Unlimited	Sparse axes store only cells with non-null attributes. Storage: O(non-null cells)
Max attributes per HyperCell	1,024	Attribute key registry limit per grid. Typical practical range: 20–150 attributes per cell
Max concurrent editors per cube	1,000	CRDT buffer capacity. Beyond 1K concurrent writers: federation node sharding recommended
Max federation peers per grid	255	VectorClock size. Each peer occupies one slot in the VectorClock map

2.6 Mapping Domain Concepts to Hypergrid Primitives

Domain Concept (Any System)	Hypergrid Primitive	Notes
An entity (portfolio item, org module, SDE)	HyperRow (all cells sharing D_1 key)	The canonical record
A property or field of an entity	D_2 key + cell.attributes['value']	Each field is one column key
A typed field value	TypedAttrValue variant	Text, Number, Json, Enum, Bool, DateTime, etc.
An entity's history over time	D_3 =TimeAxis with DimSlice	AS_OF queries
A lifecycle state	D_2 ='status' cell with LWW CRDT	Enum-typed, policy-governed
A tag / label / hashtag	D_2 ='tags' cell with OR-Set CRDT	Multi-valued, concurrent-safe
Version number	D_2 ='version' cell with MaxRegister CRDT	Always monotonically increasing
A relationship between entities	HypergraphEdge (any EdgeType)	Typed, weighted, directed
A hierarchy (parent-child)	HypergraphEdge::Hierarchy	Rollup, drill-down via graph traversal
A dependency between entities	HypergraphEdge::Dependency	With cycle detection
Permission level	PermissionTier enum on each attribute key	Per-key write access control
Audit trail	EventLog (append-only, time-indexed)	Every mutation captured
Multi-node sync	VectorClock + CrdtLog	Causal ordering guaranteed
AI-computed attribute	AttrComputation::AiEngine on any AttributeKeyDef	Score, classification, recommendation
Cross-system reference	HypergraphEdge::CrossGridLink + ShadowCell	Read-only reflection from another Grid
Namespace / address	NamespacePath URI	hypergrid://{grid}/{space_type}/{slug}/...
Time-travel query	AS_OF operator in HyperQL	Replay EventLog to any timestamp

3. Core Module Architecture

3.1 Module Inventory

Module	Code	Description
Core Substrate	HG-CORE	Universal data model: Grid, Hypercube, DimensionAxis, HyperCell, HyperRow, EventLog, CrdtLog, VectorClock, PolicyEngine
Dimension System	HG-DIM	Declare, type, index, and govern custom dimensions. DimensionAxis registry. Sparse/dense encoding. Cross-dim join planning
Cell Attribute Model	HG-CELL	N-attribute cell: attribute key registry, typed attribute values, attribute-level CRDT, computed attributes, inheritance
View Engine	HG-VIEW	HypercubeView: N-dim filter (DimSlice), sort, group, pivot, projection, DimFold, board modes, rendering adapters
Computation Engine	HG-COMP	ComputedAttribute evaluation: synchronous arithmetic tier and asynchronous AI/ML tier. Formula language. Rollup engine
Distributed Consistency	HG-CRDT	N-dim CRDT: LWW per attribute, OR-Set for set-valued, custom lattice for status, VectorClock per federation node
Graph & Network	HG-GRAPH	Hypergraph: typed edges between HyperCells/HyperRows/Grids. Link forest and tree traversal. Shadow cell protocol
Spaces & Workspaces	HG-SPACE	Bounded operational contexts (Spaces) and active working sessions (Workspaces). Governance. Treasury. Space-link sub-graph
Identity & Multi-Tenancy	HG-ID	Multi-tenant, multi-identity, multi-profile model. TenantPartition, VisibilityMask, SplitPolicy, CrossTenantMerge
Namespace System	HG-NS	Hierarchical addressing for all entities. NamespacePath URI scheme. Resolution protocol. Federation sync
Export & Integration	HG-EXPORT	CSV, XLSX, JSON, JSON-LD, Parquet, Arrow, GraphQL, webhook, streaming export of any N-dim view
AI Intelligence Layer	HG-AI	Pluggable intelligence engines: anomaly detection, health scoring, recommendation, prediction, NL-to-query translation
Query Language	HG-QL	HyperQL — N-dimensional query language: SELECT, DimSlice WHERE, FOLD, EXPAND, TRAVERSE GRAPH, AS_OF
Plugin System	HG-PLUGIN	HypercubePlugin trait for custom dimension types, attribute types, computed models, rendering adapters, and data connectors
Telemetry & Ops	HG-OPS	Distributed tracing, metrics, structured audit logs, health probes, federation sync health

3.2 The Universal Cell Store (UCS)

The Universal Cell Store is the physical storage layer that holds all HyperCells across all Hypercubes across all domain systems. It is the single source of truth for the entire Apapo platform. All domain systems read from and write to the same UCS through their respective Bridge Layers, and they all share the same EventLog, CrdtLog, and Hypergraph.

The UCS is keyed by a tuple of (grid_id, cube_id, dim_keys_tuple) — a globally unique coordinate address for every HyperCell in the system. The storage encoding depends on the declared cardinality of the Hypercube's axes: Dense Encoding stores all cells in a pre-allocated structure (for $N \leq 3$ cubes with known key sets), Sparse Encoding stores only non-null cells as key-value records (for $N \geq 4$ cubes and infinite axis key spaces), and Hybrid Encoding uses dense for the first two axes and sparse for higher dimensions.

3.3 The HyperCell Attribute Model

Attribute Key	Type	CRDT	Description
value	TypedAttrValue	LWW	The primary cell value — the 'cell content' in spreadsheet terms
type	AttrType enum	LWW	The declared type of the value: Text Number Currency Bool DateTime Enum Json Relation Geo Tag User Computed Ai
formula	Option<Formula Expr>	LWW	If this is a ComputedAttribute: the formula expression or AI engine reference
version	u64	MaxRegister	Monotonic version counter. Incremented on every mutation.
computed_by	Option<String>	LWW	If AI-computed: the engine plugin ID and model version that produced this value
computed_at	Option<DateTime>	LWW	Timestamp of last AI computation
confidence	Option<f64>	LWW	AI confidence score [0.0–1.0] for AI-derived attributes
visibility	AttrVisibility	LWW	Public Follower Connection Owner — per-attribute visibility override
tags	Vec<String>	OR-Set	User-defined tags on this attribute value
crdt_semantics	CrdtSemantics	fixed	The CRDT merge strategy for this attribute key: LWW OR-Set GrowOnlyCounter Lattice
permission_tier	PermissionTier	LWW	Minimum permission required to write this attribute: Public Member Editor Admin System
source_grid	Option<GridId>	LWW	If this is a MirrorAttribute: the Grid that is the authoritative source
last_actor	String	LWW	Identity string of the last actor to write this attribute
last_mutated_at	DateTime<Utc>	LWW	Server-assigned timestamp of the last mutation

4. CRDT Distributed Consistency (HG-CRDT)

4.1 Why Per-Attribute CRDTs?

Most distributed systems apply a single merge strategy to an entire record or document. Hypergrid takes a more granular approach: every attribute key in every HyperCell has independently configured CRDT semantics. This allows different fields of the same entity to have different conflict resolution behavior, because different fields have semantically different properties. A name field should resolve with last-write-wins. A tags set should resolve with OR-Set semantics (both concurrent additions survive). A version number should always take the maximum. A lifecycle status should follow a lattice based on the legal state transition graph.

4.2 CRDT Semantics Reference

CRDT Type	Semantics	Merge Rule	Typical Use
LastWriteWins (LWW)	Most recent timestamp wins	$\max(\text{timestamp_a}, \text{timestamp_b}) \rightarrow \text{winning value}$	Name, description, title, config, JSON blobs, any human-authored field
OR-Set (Observed-Remove Set)	Union of all concurrent additions; removals only apply to causally-known entries	$\text{additions_a} \cup \text{additions_b}$, minus removals with causal history	Tags, labels, hashtags, member lists, permission lists, dependency sets
GrowOnlyCounter	Monotonically increasing integer	$\max(\text{counter_a}, \text{counter_b})$ for bounded; $\text{sum}(\text{increments})$ for distributed counter	Restart counts, like counts, view counts, retry counts
MaxRegister	Value is always the maximum seen	$\max(\text{value_a}, \text{value_b})$	Version numbers, sequence numbers, highest-seen metrics
Lattice (Lifecycle)	State follows a partial order; only forward transitions are valid	$\text{join}(\text{state_a}, \text{state_b})$ per lattice definition — no backward transitions	ComponentStatus, SdeStatus, SolutionLifecycleState, ModuleLifecycleState

4.3 CRDT Semantics Selection Policy

All domain systems using Hypergrid adopt the following policy for assigning CRDT semantics to attribute keys:

Attribute Characteristic	CRDT Semantics	Rationale
Human-readable name, description, title	LastWriteWins	Text is authored by one person at a time; latest version wins

Attribute Characteristic	CRDT Semantics	Rationale
Lifecycle status (Draft, Active, Archived)	LastWriteWins (v1); Lattice (v2)	Business logic governs transitions; LWW safe in v1, Lattice enforces governance in v2
Multi-valued collections (tags, members)	OR-Set	Concurrent additions must both survive; removals must be causally-safe
Monotonic counters (restart count, version)	GrowOnlyCounter or MaxRegister	Counters must never decrease; max/sum merge is always safe
Binary flags (enabled, archived)	LastWriteWins	Boolean flags have no concurrent conflict semantics beyond timestamp
JSON configuration blobs	LastWriteWins	Structured JSON treated as atomic; last writer owns the full blob
AI-computed signals (health score, risk score)	LastWriteWins (system-write only)	AI engines are the sole writers; LWW on system-write is safe and simple
Numeric aggregates (budget_spent, hours_logged)	GrowOnlyCounter	Summable values only increase; GrowOnly is correct for accumulative fields
Relationship references (parent_id, owner_id)	LastWriteWins	Reference fields have single-valued semantics; LWW is correct

4.4 VectorClock and Federation Sync

Every Grid deployment maintains a VectorClock — a map from NodeId to logical timestamp (HashMap<NodeId, u64>) — that provides causal ordering of all mutations across federation peers. When two federation nodes merge their CrdtLogs, the VectorClock allows the system to determine:

- Which operations happened causally before others (and therefore need not be considered concurrent)
- Which operations are genuinely concurrent (no causal relationship) and require CRDT merge
- Whether any operations are missing from a peer's log (requiring re-sync)

Federation sync occurs over Kafka (or equivalent message broker) via a delta-sync protocol. Each node publishes CrdtLog deltas to a per-grid topic. Peer nodes consume from all connected grid topics and apply incoming deltas through the CRDT merge engine. The VectorClock ensures that even if deltas arrive out of order (due to network reordering), the merge result is always causally consistent.

The maximum federation size is 255 nodes per Grid, limited by VectorClock map size. For larger federations, hierarchical federation is supported: a Tier-1 Grid federates with Tier-2 Grids, which themselves federate with their own peers. This tree structure can scale to thousands of physically independent Grid deployments while maintaining causal consistency within each tier.

5. The Hypergraph Layer (HG-GRAPH)

5.1 Overview

The Hypergraph is Hypergrid's property graph layer. It provides typed, directed, weighted edges between any two addressable entities in the system — HyperCells, HyperRows, Hypercubes, Spaces, or entire Grids. The Hypergraph is the connective tissue of the Apapo platform: it models every relationship between entities across domain boundaries, tenant boundaries, and federation node boundaries.

5.2 Edge Type Taxonomy

EdgeType	Direction	Cardinality	Description	Domain Uses
Hierarchy	Directed (parent → child)	One-to-many	Parent-child containment relationships. Enables rollup aggregation and drill-down.	Program → Project → Task; Org → Department → Team
Dependency	Directed (A depends on B)	Many-to-many	A cannot complete before B completes. Cycle detection enforced at write time.	SDE → Library; Project → Project; Solution → Solution
Collaborates	Undirected (consent required)	Many-to-many	Collaborative relationship between entities or identities. Requires consent flow.	Kogi worker ↔ Kogi worker; SDE team members
InvestedIn	Directed (investor → asset)	Many-to-many	Economic investment or patronage relationship.	Worker → Project; Org → Solution Factory
CrossGridLink	Directed (host → source)	Many-to-many	Inter-Grid reference creating ShadowCells in the host Grid. Consent-gated.	Kogi portfolio → Qala solution; Ume org → Kogi worker
References	Directed (A → B)	Many-to-many	Informational reference without dependency or collaboration semantics.	Artifact → Source; Campaign → Product
Produces	Directed (producer → output)	One-to-many	An entity or process produces a downstream artifact or entity.	SDE → Solution; Campaign → Lead; OKR → Initiative
Governs	Directed (policy → subject)	Many-to-many	A governance rule, policy, or approval chain governs an entity.	Policy → Component; GovernanceProposal → Module

5.3 ShadowCell and MirrorAttribute Protocol

When a CrossGridLink is created between Grid A (the host) and Grid B (the source), Hypergrid automatically creates ShadowCells in Grid A that reflect the linked entity from Grid B.

ShadowCells are read-only in the host Grid — they cannot be modified by the host Grid's users. They are updated by the MirrorAttribute sync protocol whenever the source entity in Grid B changes.

Protocol Step	Actor	Description
1. Link Request	Host Grid user	User in Grid A requests to link to an entity in Grid B. CrossGridLink record created with status=Pending.
2. Consent Flow	Source Grid user	User in Grid B reviews the link request and specifies which attribute keys are permitted to be mirrored. Approves or rejects.
3. ShadowCell Creation	HG-GRAPH	On consent approval, Grid A creates a ShadowCell for the linked entity. ShadowCell is populated with the approved MirrorAttributes.
4. Delta Sync	HG-GRAPH (async)	Whenever a mirrored attribute changes in Grid B, a delta is published to the CrossGridLink channel. Grid A's shadow sync consumer applies the delta to update the ShadowCell.
5. Writeback (optional)	HG-GRAPH (governed)	If the CrossGridLink is configured with write_back_attrs, certain attributes from the host Grid A can be written back to Grid B's source entity, subject to source Grid's permission policy.

5.4 LinkForest Architecture

The LinkForest is the complete set of all CrossGridLinks and Hypergraph edges rooted at a given identity — the full picture of their inter-entity network across all Grids in the federation. Every sovereign entity (user, organization, or AI agent) has a LinkForest that can be traversed, visualized, and queried.

The LinkForest is organized as a collection of LinkTrees: each LinkTree is a rooted directed subgraph of the full Hypergraph from a single root entity to a configured maximum depth (default: 5 hops). LinkTrees are materialized on demand and cached with a configurable TTL. For large networks (>1M nodes), LinkForest traversal uses a sampled random-walk approximation for centrality computation.

6. Spaces, Workspaces, and Namespaces (HG-SPACE · HG-NS)

6.1 The Space System

A Space is a named, governed, bounded operational context within a Grid. It is a first-class Hypergrid entity — itself a HyperRow in a Space registry Hypercube — that groups a collection of Hypercubes, a member roster with permission tiers, governance configuration, and optionally a treasury for economic Spaces. Spaces provide the organizational layer that sits above raw Hypercubes but below the full Grid.

SpaceType	Description	Auto-Provisioned Cubes	Governance Model
Personal	Individual user's private operational context	components, kpis, benefits, journal	Single-owner; no governance required
Team	Small collaborative group with shared entities	components, tasks, kpis, contribution_ledger	Role-based; designated admins
Community	Open or semi-open community Space	components, announcements, governance	Elected governance; proposal voting
Organization	Formal organization with departments and treasury	all 42 Ume modules as sub-cubes	RBAC; board governance; audit trail
Project	Time-bounded project Space	components, tasks, milestones, contribution_ledger	Project lead; milestone governance
Federation	Multi-Grid collaboration Space	components, cross-portfolio, governance	Multi-Grid consensus; federated CRDT
Factory	Qala Solution Factory operational context	sdes, solutions, ccrs, releases, artifacts	Governance packs; CCR workflow

6.2 Workspace Sessions

A Workspace is an active working session within a Space. While a Space is a durable organizational context (it persists indefinitely), a Workspace is a personalized, ephemeral session configuration — the set of Hypercubes open, the current view state, the active filters and sorts, the cursor positions, and the collaboration presence of a specific user at a specific moment in time.

Workspaces can be saved and restored, enabling a user to return to exactly where they left off. Workspace templates can be shared within a Space — a team lead can define a 'Project Review' Workspace template that, when opened, shows the Projects Hypercube in Kanban mode, the KPIs cube in a time-series chart, and the Risks cube sorted by severity.

6.3 Namespace System

The Namespace System (HG-NS) provides hierarchical, globally unique addressing for every entity in the Apapo platform. Every Grid, Space, Hypercube, HyperRow, and HyperCell has a NamespacePath URI that can be resolved to the underlying entity regardless of which Grid deployment it lives on.

Entity Type	NamespacePath Pattern	Example
Grid	hypergrid://{grid_slug}/	hypergrid://kogi-platform/
Space	hypergrid://{grid}/{space_type}/{slug}/	hypergrid://kogi-platform/personal/alice/
Hypercube	hypergrid://{grid}/{space}/{cube_name}/	hypergrid://kogi-platform/personal/alice/components/
HyperRow (Entity)	hypergrid://{grid}/{space}/{cube}/{entity_id}/	hypergrid://kogi-platform/personal/alice/components/proj-uuid/
HyperCell	hypergrid://{grid}/{space}/{cube}/{entity_id}/{field_name}	hypergrid://kogi-platform/personal/alice/components/proj-uuid/health_score
Kogi worker	kogi://{user_handle}/	kogi://@alice_creator/
Kogi portfolio	kogi://portfolio/{portfolio_slug}/	kogi://portfolio/alice-design-studio/
Ume organization	ume://{org_slug}/	ume://acme-corp/
Qala SDE	qala://{factory}/{factory_slug}/sde/{sde_id}/	qala://{factory}/pharma-factory/sde/drug-app-v1/

7. HyperQL — N-Dimensional Query Language

7.1 Overview

HyperQL is Hypergrid's native query language for N-dimensional data. It extends SQL-like syntax with operations that are specific to N-dimensional grids: DimSlice (multi-axis filtering), DimFold (axis aggregation / GROUP BY generalization), DimExpand (axis pivot), TRAVERSE GRAPH (Hypergraph traversal), and AS_OF (time-travel). HyperQL is the primary query interface for all domain systems, analytical models, and AI engines.

7.2 HyperQL Syntax Reference

```
-- Basic entity query (N=2)
SELECT cell[D1.id, "field_name"].value AS field
FROM kogi.portfolio.components
WHERE cell[D1.id, "status"].value = "Active"
LIMIT 50 OFFSET 0;

-- Time-series slice (N=3 with TimeAxis)
SELECT D1.entity_id, D3.period, cell[D1, D2, D3, "metric"].value AS value
FROM kogi.portfolio.kpis
WHERE D3.period BETWEEN '2025-Q1' AND '2026-Q4'
  AND D2.metric_name = 'health_score'
ORDER BY D3.period;

-- DimFold: collapse time axis by averaging
SELECT D1.entity_id, FOLD D3 WITH AVG(cell.value) AS avg_score
FROM kogi.portfolio.kpis
WHERE D2.metric_name = 'health_score'
  AND D3.period >= '2025-Q1'
GROUP BY D1.entity_id;

-- Graph traversal
TRAVERSE GRAPH
  FROM {start_node_id}
  EDGE_TYPE = Hierarchy
  DIRECTION = Outbound
  MAX_DEPTH = 5
SELECT node.id, node.name, node.status;

-- AS_OF time-travel
SELECT *
FROM kogi.portfolio.components
AS_OF '2026-03-15T14:22:00Z'
WHERE cell[D1, "status"].value = "Active";

-- Cross-cube join via Hypergraph
SELECT a.name AS component_name, b.name AS linked_solution
FROM kogi.portfolio.components AS a
JOIN qala.solutions AS b ON GRAPH_EDGE(a.id, b.id, 'CrossGridLink')
WHERE a.status = "Active" AND b.lifecycle_state = "Active";

-- Multi-dim pivot (DimExpand)
SELECT D1.solution_id,
  EXPAND D4 AS COLUMNS(AVG(cell.value))
FROM qala.solutions.metrics
WHERE D2.metric_name = 'defect_density'
```

```
    AND D3.period >= '2026-Q1'
GROUP BY D1.solution_id;
-- Result: solution_id | security_avg | quality_avg | performance_avg

-- Shadow cell include
SELECT *, SHADOW_CELL(grid_id, coord, "health_score") AS partner_health
FROM kogi.portfolio.components
INCLUDE SHADOW CELLS FROM hypergrid://partner-grid/;
```

7.3 DimSlice Operations

DimSlice Type	Description	Example	Result
Point Slice	Exact value on one axis. Equivalent to a single row lookup (D ₁) or column lookup (D ₂).	D ₁ = project_id_42	All cells for that single project across all other axes
Range Slice	Range of values on an ordered axis. Efficient on BTree or TimeSeries indexes.	D ₃ (Time) BETWEEN 2026-01-01 AND 2026-06-30	All cells in H1 2026 across all rows and columns
Set Slice	A set of specific values on any axis.	D ₄ (Region) IN ['EMEA', 'APAC']	All cells in EMEA and APAC only
Predicate Slice	A computed condition on any attribute of cells at a given axis coordinate.	D ₂ = 'health_score' AND value > 80	All rows whose health_score > 80
Hierarchical Slice	Drill-down into a HierarchyAxis: select a node and all its descendants.	D ₅ (OrgUnit) UNDER 'Engineering'	All cells in Engineering + all sub-units
DimFold	Collapse an axis by aggregating all values along that axis into a single summary.	FOLD D ₃ (Time) WITH SUM(value)	Cube without time axis; values summed across all periods
ShadowSlice	Include cells from linked (Shadow) grids in results alongside owned cells.	INCLUDE SHADOW CELLS FROM grid://partner/	Own cells + shadow-linked cells from specified grid

8. AI Intelligence Layer (HG-AI)

8.1 Tier-1 vs Tier-2 Computed Attributes

Tier	Name	Execution	Dependencies	Latency	Examples
Tier 1	Synchronous Formula	Inline, on every read or write	Other attribute keys in the same or related HyperCells	< 1ms	budget_remaining = allocated - spent; health_score = weighted_average(kpi_scores)
Tier 2	Asynchronous AI Signal	Async, triggered by attribute change events; written back via WritebackService	Full HyperRow context + external data + historical EventLog	100ms – 30s	PortfolioHealthScore, RiskScore, AnomalyFlag, MatchScore, NarrativeRecommendation

8.2 AIEngineAdapter Plugin Interface

```
// Every AI engine in the Apapo platform implements this interface
pub trait AIEngineAdapter: Send + Sync {
    fn engine_id(&self) -> &str;
    fn name(&self) -> &str;
    fn version(&self) -> &str;

    // Which attribute keys does this engine produce?
    fn output_keys(&self) -> Vec<AttributeKey>;

    // Which attribute keys does this engine consume as inputs?
    fn input_keys(&self) -> Vec<AttributeKey>;

    // Compute one or more AI-derived attributes for a given HyperRow context
    async fn compute(
        &self,
        ctx: &AiComputeContext<'_>,
    ) -> HypergridResult<Vec<(AttributeKey, TypedAttrValue)>>;

    // Called when an input attribute changes – should we re-compute?
    fn should_recompute(
        &self,
        changed_key: &AttributeKey,
        old_value: &TypedAttrValue,
        new_value: &TypedAttrValue,
    ) -> bool;
}
```

8.3 AI Writeback Protocol

When an AI engine completes computation, it submits results via the WritebackService — a gRPC service that accepts engine-computed values, validates permissions, enforces audit trails, and writes the resulting TypedAttrValues into the appropriate HyperCells. The WritebackService enforces that:

- Only system-tier permission (PermissionTier::System) can write AI-computed attribute keys

- Every AI write is recorded in the EventLog with actor='ai::{engine_id}' and a full provenance record
- The computed_by, computed_at, and confidence attributes are always set alongside the value
- Stale AI signals are invalidated after a configurable TTL if no recomputation is triggered

8.4 Natural Language to HyperQL Bridge

The HG-AI module provides a natural language query translation layer that converts user-typed natural language queries into valid HyperQL statements. This enables non-technical users across all domain systems to perform complex multi-dimensional queries without knowing HyperQL syntax.

Natural Language Input	Generated HyperQL
Show me everything at risk this week	SELECT * FROM components WHERE risk_score > 70 OR (due_date < TODAY+7 AND status = 'Active')
What were my top 3 projects by revenue last quarter	SELECT name, revenue FROM kogi.components WHERE type='Project' AND D3.period='2026-Q1' ORDER BY revenue DESC LIMIT 3
List all modules in the finance domain with health below 60	SELECT * FROM ume.kernel.modules WHERE domain_area='Finance' AND health_score < 60
How many solutions shipped in Q4 2025 vs Q4 2026	SELECT D3.period, COUNT(*) FROM qala.solutions WHERE lifecycle_state='Released' AND D3.period IN ('2025-Q4','2026-Q4') GROUP BY D3.period

9. Security, Governance, and Compliance

9.1 Permission Tier Model

Permission Tier	Description	Can Write	Can Read
Public	No authentication required	No — read-only access to public attributes	All attributes marked Public visibility
Member	Authenticated member of the Space	User-level fields: name, description, tags, status	All non-private attributes
Editor	Named editor with elevated access	All Member fields + configuration fields	All non-owner attributes
Admin	Space administrator	All Editor fields + governance fields + member roster	All attributes including audit trail
System	Internal system actor (AI engine, federation sync)	AI-computed attributes + federation sync fields	All attributes including encrypted fields
Owner	Sovereign Entity that owns the Grid	All attributes with no restriction	All attributes with no restriction

9.2 Row-Level Security

Hypergrid enforces row-level security at the storage layer through `VisibilityMask` — a set of N-dimensional `DimSlice` predicates that define what is visible to each observer type. `VisibilityMasks` are evaluated on every read operation, before results are returned to the caller. The `PolicyEngine` enforces that no read path can bypass `VisibilityMask` evaluation.

9.3 EventLog as Immutable Audit Trail

Every mutation to any HyperCell attribute in the Universal Cell Store produces an `EventLog` entry. The `EventLog` is append-only and immutable — entries can never be modified or deleted. Every entry records the actor identity, the timestamp, the before-value, the after-value, the CRDT operation applied, and the `VectorClock` state at the time of the mutation.

The `EventLog` serves multiple functions simultaneously: it is the basis for `AS_OF` time-travel queries (replaying entries to reconstruct historical state), the basis for CRDT delta sync between federation nodes (distributing deltas via `CrdtLog`), the basis for AI engine event subscriptions (triggering recomputation), and the basis for compliance audit exports.

9.4 GDPR/CCPA Compliance Automation

Compliance Requirement	Hypergrid Mechanism
Right to Erasure (GDPR Art. 17)	SovereignEntity-level archive + personal data attribute tombstoning. The EventLog retains the audit record; the attribute values are replaced with {GDPR_ERASED} markers.
Data Portability (GDPR Art. 20)	HG-EXPORT Parquet/JSON-LD export of all HyperCubes owned by an identity. Includes full EventLog and complete attribute history.
Consent Tracking	CrossGridLink consent records stored as immutable HyperRows with consent timestamp, consented attribute list, and consent withdrawal timestamp.
Data Residency	GeoAxis on TenantAxis enforces that certain Hypercubes are only stored on federation nodes in specified geographic regions (GDPR adequacy regions).
Audit Trail	Immutable EventLog with cryptographic hash chaining (optional). Hash chain provides tamper-evidence for regulatory submissions.

10. The Domain-Layer Pattern

10.1 Architecture Overview

Every domain system built on Hypergrid follows the same structural pattern — the Domain-Layer Pattern. It consists of four stacked layers, each with a clear responsibility boundary:

PRESENTATION LAYER Domain-specific UI · CLI · REST API · WebSocket · SDK
DOMAIN LOGIC LAYER Domain entities · Business rules · Lifecycle state machines Governance policies · Analytical models · Domain workflows
DOMAIN-HYPERGRID BRIDGE LAYER (the "Store" or "Codec") Entity ↔ HyperCell serialization · Schema registration Attribute key definitions · CRDT semantics config Graph edge type mappings · AI engine registration
HYPERGRID SUBSTRATE LAYER Grid · Hypercube · UniversalCellStore · Hypergraph VectorClock · CrdtLog · EventLog · HyperQL · AI Engine NamespaceSystem · SpaceSystem · Federation

10.2 The Codec Contract

```
pub trait DomainStore<Entity, EntityId>: Send + Sync {
    /// Register all Hypercubes and attribute schemas on startup.
    fn bootstrap(grid: &mut Grid) -> HypergridResult<CubeId>;

    /// Write a domain entity into the Hypercube as a set of HyperCells.
    fn write(grid: &mut Grid, cube_id: CubeId, entity: &Entity, actor: &str)
        -> HypergridResult<()>;

    /// Read all HyperCells for an EntityId and reconstruct the domain entity.
    fn read(grid: &Grid, cube_id: CubeId, id: EntityId)
        -> DomainResult<Option<Entity>>;

    /// Check whether an entity exists in the Hypercube.
    fn exists(grid: &Grid, cube_id: CubeId, id: EntityId) -> bool;
}
```

The codec contract enforces complete separation between domain model and storage substrate. The domain entity type (Component, OrgModule, Solution) is a pure Rust struct with no storage concerns. The Hypercube stores it as a set of HyperCells. The codec translates between the two representations without leaking Hypergrid primitives into the domain layer.

10.3 Standard Dimension Convention

All domain systems using Hypergrid adopt the following N=2 baseline convention for their primary entity Hypercubes:

```

D1 = EntityAxis (key_type: Uuid)           ← entity row (entity ID)
D2 = PropertyAxis (key_type: String)       ← property column (field name)

cell at (entity_id, "field_name") → cell.attributes["value"] = <TypedAttrValue>

Domain systems may extend to N>2 for specific analytical Hypercubes:
- Time-series KPI cube:      D3 = TimeAxis
- Multi-tenant shared cube:  D3 = TenantAxis
- Scenario/planning cube:    D3 = ScenarioAxis, D4 = TimeAxis
- Org-hierarchy analytics:   D3 = TimeAxis, D4 = HierarchyAxis

```

Part III — Kogi: Independent Worker Operating System

11. Kogi Platform Overview

Kogi is a software-defined Independent Worker Operating System (IW-OS) built on the Hypergrid substrate. It provides a solo worker, freelancer, creative professional, entrepreneur, or independent contractor with a single unified platform to manage every dimension of their working and creative life: projects, programs, finances, resources, artifacts, gigs, benefits, collaborations, income, governance, and career development.

Design Principle: Every entity in the Kogi ecosystem — programs, projects, resources, assets, artifacts, gigs, benefits, finances, relationships, governance actions, analytics events — is a row in one universal, scalable, configurable spreadsheet. The Portfolio Management System is that spreadsheet: the master, living, computed record of an independent worker's entire operational and creative world.

11.1 Kogi Module Codes

Module Code	Module Name	Responsibility
KPMS	Portfolio Management System	Master orchestration layer: spreadsheet substrate, row/column model, sheet registry, view engine
KPSS	Portfolio Spreadsheet Substrate	Low-level Rust data engine: PortfolioRow, ColumnSchema, SheetDefinition, indexed CellStore
KPRG	Portfolio Component Registry	Canonical registry of all PortfolioComponents system-wide; the master index
KPVW	View Engine	Filter · Sort · Group · Pivot · Slice across any column dimension; 30+ board modes
KPCM	Computation Engine	20+ analytical models; derived column computation; rollup and aggregation
KPCL	Collaborative Editing	CRDT-backed concurrent multi-user spreadsheet editing with contribution attribution
KPEX	Export & Integration	CSV · XLSX · JSON · API · Embed · Webhook export of any sheet or view
KLNK	Link Network	Inter-portfolio graph system: forests, trees, ShadowRows, MirrorColumns, cross-user connections
KPID	Portfolio Identity System	Multi-account, multi-identity, multi-profile management; VisibilityMask; SplitPolicy
KSPC	Space Module	Named, governed, bounded digital environments for portfolios and identities

Module Code	Module Name	Responsibility
KWSP	Workspace Module	Active working context within a Space: open sheets, session state, tools, presence
KNSPC	Namespace Module	Hierarchical addressing for all platform entities via NamespacePath URIs

12. Kogi Data Model on Hypergrid

12.1 The PortfolioComponent as HyperRow

The fundamental unit of the Kogi data model is the PortfolioComponent — a universal node that represents any entity in the Kogi ecosystem. Every PortfolioComponent is stored as a HyperRow in the `kogi.portfolio.components` Hypercube: a set of HyperCells sharing the same D_1 key (the component's UUID), with each cell representing one field of the component.

Kogi Spreadsheet Concept	Hypergrid Mapping
PortfolioSystem (workbook)	A Grid containing all Hypercubes for one sovereign entity's entire portfolio
Sheet (Projects Sheet, Finances Sheet)	A HypercubeView — a named DimSlice + column projection over the primary components cube
PortfolioRow (row)	A HyperRow — all HyperCells sharing the same D_1 entity key
PortfolioColumn (column)	A D_2 attribute key + its TypedAttrValue and CRDT semantics configuration
Cell value	<code>cell(entity_id, field_name).attributes['value']</code> — a TypedAttrValue
ComputedColumn (formula)	<code>AttrComputation::Formula</code> — a Tier-1 synchronous computed attribute
AIColumn (engine signal)	<code>AttrComputation::AiEngine</code> — a Tier-2 async AI-computed attribute
SheetDefinition	A saved HypercubeView configuration: DimSlice predicates + column ordering + render mode
ViewDefinition	A user-saved customization of a HypercubeView with additional filters and layout preferences

12.2 Primary Hypercubes in Kogi

Cube Name	Dimensions	Primary Entities	Key D_2 Fields
<code>kogi.portfolio.components</code>	N=2 (Entity, Property)	All Component nodes (Items and Containers)	<code>id</code> , <code>name</code> , <code>type</code> , <code>status</code> , <code>parent_id</code> , <code>owner_id</code> , <code>created_at</code> , <code>tags</code>

Cube Name	Dimensions	Primary Entities	Key D ₂ Fields
kogi.portfolio.kpis	N=3 (Entity, Metric, Time)	KPI measurements over time periods	health_score, risk_score, alignment_score, velocity, budget_utilization
kogi.portfolio.finances	N=3 (Entity, Account, Time)	Financial records: income, expenses, allocations	amount, currency, category, direction, counterparty, balance
kogi.portfolio.benefits	N=2 (Entity, Property)	Benefits coverage: health, dental, retirement, insurance	provider, plan_type, coverage_start, coverage_end, premium, status
kogi.portfolio.actions	N=2 (Entity, Property)	ActionKind event records for the activity log	action_type, actor, target_id, timestamp, metadata
kogi.portfolio.approvals	N=2 (Entity, Property)	ApprovalRequest records for governance workflows	requester, approvers, status, deadline, decision, rationale
kogi.portfolio.collaborations	N=2 (Entity, Property)	Active collaboration records with other workers	collaborator_id, role, contribution_type, visibility, consent_granted
kogi.portfolio.links	N=2 (Entity, Property)	KLNK InterPortfolioLink records	source_id, target_id, edge_type, weight, consent_status, mirror_attrs

12.3 Component Type Taxonomy

ItemType	Description	Typical Parent Container
Program	A strategic initiative grouping multiple projects	Root or another Program
Project	A bounded effort with deliverables and timeline	Program
Task	An atomic unit of work within a Project	Project or Sprint
Artifact	A creative or technical output: document, design, codebase	Project or Resource
Resource	A reusable asset: person, tool, budget pool, equipment	Program or Portfolio
Gig	A contracted piece of work: freelance engagement, booking	Program or Resource
Benefit	A coverage entitlement: health, insurance, retirement	Personal Space

ItemType	Description	Typical Parent Container
Grant	A funding opportunity or awarded grant	Program
Metric	A KPI measurement or analytical data point	Any container
GovernanceProposal	A formal governance action requiring member vote	Organization Space
CrowdresourcingCampaign	A campaign soliciting contributions from the community	Community Space
MarketplaceListing	A published listing on the Kogi Marketplace	Resource or Artifact

13. Kogi AI Engine — Oba and kogi-engine

13.1 kogi-engine Architecture

kogi-engine is the Scala 3 analytics and intelligence engine that consumes all portfolio events from the Kogi EventLog and writes computed signals back into the portfolio via the WritebackService. It operates as a streaming computation engine with 20+ sub-engines, each responsible for a specific class of AI-derived columns.

AI-Computed Attribute	Hypercube	Engine	Update Trigger
health_score	kogi.portfolio.kpis	KogiHealthScoreEngine	On any component mutation
risk_score	kogi.portfolio.kpis	KogiRiskEngine	On risk record change or deadline approach
alignment_score	kogi.portfolio.kpis	KogiAlignmentEngine	On program or strategy mutation
collaboration_score	kogi.portfolio.kpis	KogiCollaborationEngine	On contribution event
maturity_score	kogi.portfolio.components	KogiMaturityEngine	On artifact change
anomaly_flag	kogi.portfolio.components	KogiAnomalyEngine	On metric deviation beyond 2σ
match_score	kogi.portfolio.collaborations	KogiMatchEngine	On skill or requirement update

AI-Computed Attribute	Hypercube	Engine	Update Trigger
income_projection_90d	kogi.portfolio.finances	KogiIncomeProjectionEngine	On gig or contract change
budget_burn_rate	kogi.portfolio.finances	KogiBudgetEngine	On financial transaction
narrative_summary	kogi.portfolio.components	KogiNarrativeEngine	On significant milestone completion

13.2 Oba AI Integration

Oba is the Kogi platform AI assistant — the spreadsheet's AI chief-of-staff. It reads every row, every column, and every engine signal, and surfaces proactive, context-aware intelligence directly in the spreadsheet UI. Oba operates in three modes:

Oba Mode	Description	Examples
Reactive	Responds to explicit user queries in natural language. Translates NL to HyperQL. Answers questions about the portfolio.	'Show me everything at risk this week' → executes filtered HyperQL query
Proactive	Surfaces insights, flags anomalies, and suggests actions without being asked. Reads engine signals and EventLog.	Flags: 'This project is 14 days behind schedule — adjust due date or reduce scope?'
Autonomous (approval-first)	Executes bulk actions, generates artifacts, prepares proposals for user approval. Never executes without confirmation.	'Archive all completed projects older than 90 days' → prepares batch action for user review

14. KLNK — The Kogi Link Network

14.1 Overview

KLNK is the inter-portfolio graph system that maps the full network of connections between Kogi users, portfolios, organizations, and federation nodes. Every connection between two workers, between a worker and an organization, between a portfolio and a marketplace listing, between a gig worker and a client — is a LinkEdge in the KLNK graph. The full graph of all LinkEdges and all connected nodes is the global Kogi economic graph.

14.2 LinkEdge Types

LinkEdgeType	Direction	Consent Required	Description
Collaborates	Undirected	Yes — mutual	Active collaboration relationship between two workers on a shared component
InvestedIn	Directed (A → B)	Yes — from B	Worker A has invested in or patronized Worker B's work or organization
Employs	Directed (org → worker)	Yes — from worker	An organization employs or contracts a worker
Follows	Directed (A → B)	No	Worker A follows Worker B's public portfolio updates
Endorses	Directed (A → B)	No	Worker A endorses a skill or work product of Worker B
SharedPortfolio	Directed (owner → contributor)	Yes — from contributor	A contributor has access to a shared portfolio or cooperative
MarketplaceTransaction	Directed (buyer → seller)	Via transaction record	A completed marketplace transaction between two entities
Federation	Undirected	Grid-level trust	Two independent Grid deployments federated for CRDT sync

14.3 ShadowRow Protocol

When Worker A creates a Collaborates or InvestedIn link to Worker B, and Worker B grants consent with a configured attribute visibility list, Hypergrid creates a ShadowRow in Worker A's `kogi.portfolio.components` cube that mirrors the linked component from Worker B's cube. The ShadowRow is read-only in Worker A's cube and is kept synchronized via the MirrorAttribute delta-sync protocol.

15. Kogi Spaces Configuration

Kogi Context	HG-SPACE SpaceType	Slug Pattern	Auto-Provisioned Cubes
Personal Portfolio	Personal	<code>kogi://worker/{user_id}/personal/</code>	components, kpis, benefits, finances
Shared Portfolio	Team	<code>kogi://portfolio/{portfolio_slug}/</code>	components, contribution_ledger, kpis
Cooperative Portfolio	Community	<code>kogi://cooperative/{coop_slug}/</code>	components, governance, kpis, treasury
Federation Portfolio	Federation	<code>kogi://federation/{fed_slug}/</code>	components, cross-portfolio, governance

Kogi Context	HG-SPACE SpaceType	Slug Pattern	Auto-Provisioned Cubes
Crowdresourcing Campaign	Project	kogi://campaign/{campaign_ slug}/	components, contribution_ledger, milestones

Part IV — Ume: Business Operating System

16. Ume Platform Overview

Ume is a software-defined Organization / Business Operating System (B-OS) built on the Hypergrid substrate. It models a complete organization as a kernel-managed collection of 42 domain subsystems, each governing one domain area of the organization's operations. The Ume kernel supervises all modules, manages the event bus, enforces RBAC, and provides shared services.

When Ume adopts Hypergrid as its substrate, the mapping is conceptually powerful: each of the 42 organization modules becomes a domain-specific Hypercube in the Ume Grid. The kernel's module registry becomes a HyperRow in a `ume.kernel.modules` Hypercube. The event bus maps to Hypergrid's EventLog and CrdtLog. RBAC enforcement maps to Hypergrid's PermissionTier and AttrVisibility. The Supervisor maps to Hypergrid's GovernanceEngine.

17. Ume Kernel Module Hypercube

17.1 ume.kernel.modules — N=2 Module Registry

```
D1 = EntityAxis (Uuid)      ← ModuleId
D2 = PropertyAxis (String) ← module field name

Attribute Key Registry:
```

D ₂ Key	TypedAttrValue	CRDT	Notes
module_id	Text	LWW	Canonical module identifier
name	Text	LWW	Display name (e.g., 'Finance & Accounting')
domain_area	Json	LWW	DomainArea enum value
version	Text	MaxRegister	Module version string — always the maximum seen
lifecycle_state	Json	LWW	LifecycleState: Registered → Starting → Running → Degraded → Stopped
status	Json	LWW	ModuleStatus: Active Suspended Disabled
dependencies	Json	OR-Set	Vec<ModuleId> that this module depends on
restart_count	Number	GrowOnlyCounter	Supervisor restart counter — only increases

D ₂ Key	TypedAttrValue	CRDT	Notes
last_error	Json	LWW	Option<KernelError> from most recent failure
health_score	Number	LWW (AI-write)	AI-computed module health score 0–100
evaluation_cache	Json	LWW	Cached evaluation output from last evaluation run
config	Json	LWW	Module configuration blob
metrics	Json	LWW	MetricPoint observations snapshot
created_at	Json	LWW	DateTime<Utc>
updated_at	Json	LWW	DateTime<Utc>

18. The 42 Organization Module Hypercubes

#	Module Name	Primary Hypercube	Extra Dimensions
0 1	Organization Administration	ume.admin.org_units	D ₃ =TimeAxis (historical org charts)
0 2	Organization Analytics	ume.analytics.kpis	D ₃ =TimeAxis, D ₄ =DomainAxis
0 3	Backup & Recovery	ume.backup.jobs	D ₃ =CategoryAxis (backup_type)
0 4	Board Management	ume.board.meetings	D ₃ =TimeAxis
0 5	Business Development	ume.bizdev.opportunities	D ₃ =CategoryAxis (opportunity_type)
0 6	Enterprise CMS	ume.cms.content	D ₃ =TimeAxis (version history)
0 7	Communications	ume.comms.channels	D ₃ =CategoryAxis (channel_type)
0 8	CRM	ume.crm.contacts	D ₃ =CategoryAxis (contact_type)
0 9	Design System	ume.design.tokens	D ₃ =TimeAxis (token version history)
1 0	Engineering & Technology	ume.eng.projects	D ₃ =TimeAxis (sprint data)
1 1	ESG / CSR / Sustainability	ume.esg.metrics	D ₃ =TimeAxis, D ₄ =CategoryAxis

#	Module Name	Primary Hypercube	Extra Dimensions
1 2	Enterprise Engineering Admin	ume.enterprise.systems	—
1 3	Legal Entity (Chombo)	ume.chombo.entities	D ₃ =CategoryAxis (jurisdiction)
1 4	Finance & Accounting	ume.finance.accounts	D ₃ =TimeAxis (period balances)
1 5	GRC (Governance, Risk, Compliance)	ume.grc.risks	D ₃ =TimeAxis (risk history)
1 6	Human Resources	ume.hr.employees	D ₃ =TimeAxis (headcount history)
1 7	Investment Management	ume.investment.portfolio	D ₃ =TimeAxis, D ₄ =CategoryAxis
1 8	IT & Asset Management	ume.it.assets	D ₃ =CategoryAxis (asset_type)
1 9	Enterprise Knowledge	ume.knowledge.articles	D ₃ =TimeAxis
2 0	Learning & Development	ume.learning.programs	D ₃ =TimeAxis
2 1	Management & Strategy	ume.strategy.okrs	D ₃ =TimeAxis (OKR periods)
2 2	Marketing (Soko)	ume.soko.campaigns	D ₃ =TimeAxis, D ₄ =CategoryAxis
2 3	Master Data Management	ume.mdm.entities	D ₃ =CategoryAxis (entity_type)
2 4	Office & Facility	ume.facilities.sites	D ₃ =CategoryAxis (site_type)
2 5	Operations Management	ume.ops.processes	D ₃ =TimeAxis
2 6	Portal / Hub / Dashboard	ume.portal.dashboards	—
2 7	Portfolio & Program	ume.portfolio.programs	D ₃ =TimeAxis
2 8	PR & Branding	ume.branding.assets	D ₃ =TimeAxis
2 9	Process, Orchestration & Workflow	ume.process.workflows	D ₃ =TimeAxis
3 0	Product, Services & Solutions	ume.product.catalog	D ₃ =TimeAxis, D ₄ =CategoryAxis
3 1	Project Management	ume.project.projects	D ₃ =TimeAxis (sprint data)

#	Module Name	Primary Hypercube	Extra Dimensions
3 2	Procurement & Supply Chain	ume.procurement.vendors	D ₃ =CategoryAxis
3 3	R&D and Innovation	ume.rd.experiments	D ₃ =TimeAxis, D ₄ =CategoryAxis
3 4	Recruiting	ume.recruiting.candidates	D ₃ =TimeAxis (pipeline stages)
3 5	Reporting & Dashboards	ume.reporting.reports	D ₃ =TimeAxis
3 6	Resource Management	ume.resources.allocations	D ₃ =TimeAxis, D ₄ =CategoryAxis
3 7	Revenue Operations	ume.revops.pipeline	D ₃ =TimeAxis
3 8	Sales	ume.sales.opportunities	D ₃ =TimeAxis (forecast periods)
3 9	Security	ume.security.incidents	D ₃ =TimeAxis
4 0	Stakeholder Management	ume.stakeholders.registry	—
4 1	Talent Management	ume.talent.profiles	D ₃ =TimeAxis
4 2	Treasury Management	ume.treasury.accounts	D ₃ =TimeAxis (fiscal periods)

Part V — Qala: Solution Factory OS

19. Qala Platform Overview

Qala is a Solution Factory Operating System — a platform for building, governing, distributing, and monetizing solutions of any kind: software, hardware, pharmaceutical formulations, research datasets, creative works, legal templates, financial instruments, and any other formally governed, versioned, distributable artifact. Qala provides the infrastructure for organizations and individuals who produce solutions at scale.

The core metaphor in Qala is the factory: a Solution Factory is an entity that operates one or more Solution Development Environments (SDEs), within which solutions are designed, built, tested, governed, and released. The factory applies governance packs (domain-specific rules, compliance requirements, and review workflows) to ensure every solution meets the quality and regulatory standards of its domain before it can be released.

20. Qala Hypercube Inventory

Cube Name	N	Primary Entities	Key Fields
qala.factories	2	Solution Factory records	factory_id, name, domain_packs, governance_config, status, owner_org_id
qala.sdes	2	Solution Development Environment records	sde_id, factory_id, name, status, team_members, toolchain_id, governance_pack_ids
qala.solutions	3 (Entity, Property, Time)	Solution records (all types and versions)	solution_id, sde_id, type, version, lifecycle_state, quality_score, defect_density
qala.ccrs	2	Change Control Request records	ccr_id, solution_id, change_type, risk_level, status, approvers, decision_timestamp
qala.releases	2	Release records	release_id, solution_id, version, release_type, distribution_channels, release_notes

Cube Name	N	Primary Entities	Key Fields
qala.artifacts	2	Artifact records (build, test, compliance)	artifact_id, solution_id, artifact_type, hash, created_at, signed_by
qala.toolchains	2	Toolchain configuration records	toolchain_id, name, tools, versions, environment_type, validated_by
qala.ai_recommendations	2	AI Agent recommendation records	rec_id, solution_id, recommendation_type, confidence, action_proposed, status
qala.solutions.metrics	4 (Entity, Metric, Time, Domain)	Solution quality metrics over time and domain	quality_score, defect_density, compliance_score, security_score, performance_score

21. Qala Solution Lifecycle

Lifecycle State	Description	Allowed Transitions	Governance Gate
Draft	Initial creation; work in progress; no governance constraints	Draft → InReview	None
InReview	Formal review process initiated; reviewers assigned	InReview → Approved Rejected Draft	Minimum N approvers required
Approved	All required approvals obtained; ready for release staging	Approved → Staging Draft (if changes required)	All required CCRs resolved
Staging	Deployed to staging environment for pre-release validation	Staging → Released Draft (if staging fails)	Automated test suite must pass
Released	Publicly released and distributed	Released → Deprecated	Release governance pack sign-off

Lifecycle State	Description	Allowed Transitions	Governance Gate
Deprecated	End-of-life; no new consumers; existing consumers flagged	Deprecated → Archived	Deprecation notice period elapsed
Archived	Read-only historical record; no further modifications	No further transitions	Immutable; EventLog sealed

22. Qala Governance Packs

A Governance Pack is a domain-specific configuration bundle that defines the rules, review workflows, compliance checklists, and release criteria that apply to solutions in a particular regulatory domain. Governance Packs are implemented as HypercubePlugin extensions and can be composed — a solution can be governed by multiple packs simultaneously (e.g., a pharmaceutical software solution might be governed by both a Software Governance Pack and a Pharma Regulatory Pack).

Governance Pack Domain	Key Rules	Required Reviewers	AI Integration
Software (Default)	Code review, test coverage >80%, security scan, dependency audit	2 senior engineers + 1 security reviewer	Defect density prediction, security vulnerability scoring
Pharmaceutical	GMP compliance, ingredient validation, stability data review, regulatory submission tracking	QA Lead + Regulatory Affairs + Clinical Reviewer	Regulatory risk score (regulatory_risk_v2 model)
Financial Instrument	KYC/AML compliance, risk model validation, regulatory capital review, stress test results	Risk Officer + Compliance + Legal + External Auditor	Risk model validation score, exposure analysis
Medical Device (ISO 13485)	Design history file, verification and validation, risk management (ISO 14971)	Quality Manager + Regulatory Affairs + Clinical Expert	Adverse event prediction, safety signal detection
Creative Work	IP ownership verification, license validation, attribution chain review	Rights holder + IP Counsel	Similarity score (copyright check), attribution graph analysis

23. Qala AI Integration

AI-Computed Attribute	Cube	Engine	Description
quality_score	qala.solutions	QalaQualityEngine	Composite quality signal across defect density, test coverage, review thoroughness, and deployment stability
defect_density	qala.solutions	QalaDefectEngine	Predicted defect density based on code metrics, historical defect rates, and team velocity
compliance_score	qala.solutions	QalaComplianceEngine	Compliance coverage score across all active governance pack requirements
security_score	qala.solutions	QalaSecurityEngine	Security posture score: CVE exposure, dependency freshness, auth pattern analysis
regulatory_risk_score	qala.solutions	QalaRegulatoryEngine	Domain-specific regulatory risk signal (highest for pharma/finance/medical device domains)
release_readiness	qala.releases	QalaReleaseEngine	Binary + confidence score for whether a solution is ready to release based on all governance criteria

Part VI — Cross-Platform Integration & Federation

24. Cross-System Data Gravity

Because Kogi, Ume, and Qala all store their entities in the same Universal Cell Store — with different domain prefixes but identical underlying storage infrastructure — cross-system queries and integrations are first-class operations rather than engineering challenges.

A HyperQL query can join a Kogi portfolio component to a Qala solution via a CrossGridLink edge, and then enrich that view with Ume organizational data from the worker's employer record — all in a single query plan, evaluated against a single storage engine, without ETL pipelines or data warehousing.

Cross-System Scenario	HyperQL Pattern	Notes
Worker's portfolio linked to a Qala solution they contributed to	JOIN kogi.portfolio.components TO qala.solutions ON GRAPH_EDGE('CrossGridLink')	Worker must have consent from Factory; ShadowCell created in Kogi cube
Organization's employees linked to their Kogi portfolios	JOIN ume.hr.employees TO kogi.portfolio.components ON GRAPH_EDGE('Employs')	Ume employer creates CrossGridLink to worker's Kogi portfolio with consent
Qala solution attributed to a Kogi worker's portfolio	TRAVERSE GRAPH FROM qala.solutions EDGE_TYPE=Produces DIRECTION=Inbound	Kogi worker is linked as a producer of the solution
Ume organization's solutions produced by their Solution Factory	JOIN ume.product.catalog TO qala.solutions ON GRAPH_EDGE('Produces')	Organization owns or is affiliated with a Qala Factory

25. Multi-System Federation Topology

25.1 Single-Grid Deployment

In the simplest deployment, all three domain systems (Kogi, Ume, Qala) share a single Grid deployment with a single Universal Cell Store. All entities exist in the same physical storage layer. Cross-system HyperQL queries are local joins. Federation sync is only needed for multi-node horizontal scaling within this single Grid.

25.2 Multi-Grid Federated Deployment

In a production federated deployment, each domain system may run on its own Grid deployment, with CrossGridLinks and CrdtLog delta sync connecting them. This provides:

- Independent scaling: Kogi, Ume, and Qala can be scaled independently based on their workloads
- Independent governance: each domain system can enforce its own access control policies without interference
- Organizational isolation: an enterprise may run its own Ume Grid while its employees' Kogi portfolios live on the shared Kogi platform Grid
- Regulatory compliance: a pharmaceutical solution factory can run its Qala Grid in a jurisdiction-compliant data center while linked to an enterprise Ume Grid in a different region

25.3 Federation Sync Architecture

Sync Type	Protocol	Granularity	Latency
Within-Grid multi-node	CrdtLog delta via Kafka, VectorClock causal ordering	Per-attribute delta	< 100ms
Cross-Grid CrossGridLink	MirrorAttribute delta sync via CrossGridLink channel	Per-approved-attribute delta	< 1s (near-real-time)
Cross-Grid full federation	CrdtLog delta sync via Kafka (Grid-to-Grid topic)	Per-Hypercube delta	< 1s (near-real-time)
Cross-Grid bulk sync (initial)	Snapshot export (Parquet) + replay	Full Hypercube snapshot	Minutes to hours depending on size

26. Multi-Identity and Tenant Model

26.1 The SovereignEntity Model

Every participant in the Apapo platform is represented by a **SovereignEntity** — the real-world person, organization, collective, cooperative, or AI agent that owns a root set of Hypercubes and Spaces. A **SovereignEntity** can have multiple **Identities** (multiple **@handles** or organizational identities), each with its own **ProfilePartition** of the entity's data.

Identity Layer	Description	Hypergrid Mechanism
SovereignEntity	The real-world owner. Has full ownership of all data across all their identities.	Root Grid owner; PermissionTier::Owner across all cubes
Identity (Handle)	A named public-facing identity (@handle or org name). Has its own profile partition.	ProfilePartition : rows tagged with identity_id ; VisibilityMask per identity
Profile	A contextual presentation of an Identity for a specific audience (public, professional, personal).	HypercubeView DimSlice filter on profile_id attribute
Account	A login credential set associated with an Identity. Multiple accounts can link to the same Identity.	Authentication record; linked to Identity via secure token

26.2 VisibilityMask

The **VisibilityMask** is a set of N-dimensional **DimSlice** predicates that define, for each observer type (Public, Follower, Connection, Owner), exactly which rows and which attribute keys are visible. **VisibilityMasks** are evaluated at the **HG-CRDT** layer — before any data is returned from the **Universal Cell Store** — ensuring that no read path can bypass privacy controls.

Observer Type	Visibility Level	Typical Access
Public	Unauthenticated internet access	Only rows and attributes explicitly marked Public visibility
Follower	Authenticated user who follows this identity	Public + Follower-visible attributes (e.g., recent activity feed)
Connection	A user with an established CrossGridLink relationship	Follower + Connection-visible attributes (e.g., contact details, collaboration history)
Member	A member of the same Space	Connection + Space-scoped attributes (e.g., internal project details)

Observer Type	Visibility Level	Typical Access
Editor	An editor-level collaborator in the Space	Member + editor-level attributes (e.g., budget details, configuration)
Admin	A Space administrator	Editor + admin attributes (e.g., governance records, full audit trail)
Owner	The SovereignEntity that owns the cube	All attributes including encrypted and system-level

Part VII — Deployment, Persistence & Performance

27. Persistence Architecture

27.1 Primary Storage Backends

Storage Backend	Use Case	Status	Notes
PostgreSQL 16+	Primary OLTP cell store for N≤4 cubes	Production-ready	Row-level security enforced at DB layer. BRIN indexes for TimeAxis. GIN indexes for JSONB attributes.
ClickHouse	Analytics backend for N>4 cubes and high-throughput aggregation workloads	Integration v1.1	Receives Kafka delta stream from primary PG store. Used for DimFold/DimExpand on large cubes.
Redis	CrdtLog buffer for in-flight operations; session cache for Workspace state	Production-ready	Volatile; durable CrdtLog is PostgreSQL. Redis is the hot-path write buffer only.
S3 / Object Store	EventLog cold archive; Parquet snapshot exports; AI model artifact storage	Production-ready	EventLog entries are flushed to S3 after 10K-event cap in hot PostgreSQL EventLog.
Neo4j (v2.5+)	Global Hypergraph backend for large-scale graph traversal (KLNK at platform scale)	Planned v2.5	For KLNK graphs >1M nodes, Neo4j provides path traversal and centrality computation at scale.
SQLite	Local single-user deployment; offline mode	Development / Edge	Used for kogi-local desktop deployments and offline-first mobile clients.

27.2 Cell Store Physical Schema (PostgreSQL)

```
-- Primary cell store table
CREATE TABLE hypergrid_cells (
  grid_id      UUID NOT NULL,
  cube_id      UUID NOT NULL,
  dim1_key     UUID NOT NULL,           -- EntityAxis key
  dim2_key     TEXT NOT NULL,          -- PropertyAxis key
  dim3_key     TEXT,                   -- Optional TimeAxis / CategoryAxis
  key
  dim4_key     TEXT,                   -- Optional GeoAxis / TenantAxis
  key
  -- dim5..dim16_key follow the same pattern for higher-dim cubes
  attr_value   JSONB NOT NULL DEFAULT '{}', -- TypedAttrValue as JSONB
  attr_version BIGINT NOT NULL DEFAULT 0,
  attr_crdt_ts BIGINT NOT NULL,         -- Logical clock timestamp (LWW)
```

```

    last_actor          TEXT NOT NULL,
    last_mutated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (grid_id, cube_id, dim1_key, dim2_key, dim3_key, dim4_key)
);

-- EventLog (append-only, never updated)
CREATE TABLE hypergrid_events (
    event_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    grid_id            UUID NOT NULL,
    cube_id            UUID NOT NULL,
    dim_keys            JSONB NOT NULL,          -- Full N-dim coordinate tuple
    attr_key            TEXT NOT NULL,
    before_value        JSONB,
    after_value         JSONB NOT NULL,
    crdt_op             TEXT NOT NULL,          -- LWW | OR-Set-Add | OR-Set-Remove
    | Counter | Lattice
    vector_clock        JSONB NOT NULL,
    actor              TEXT NOT NULL,
    timestamp           TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- CrdtLog (in-flight operations buffer)
CREATE TABLE hypergrid_crdt_log (
    op_id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    grid_id            UUID NOT NULL,
    cube_id            UUID NOT NULL,
    dim_keys            JSONB NOT NULL,
    attr_key            TEXT NOT NULL,
    op_type             TEXT NOT NULL,
    op_value            JSONB NOT NULL,
    vector_clock        JSONB NOT NULL,
    applied             BOOLEAN NOT NULL DEFAULT FALSE,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

28. Performance Characteristics

28.1 Read/Write Latency Targets

Operation	Target Latency (p50)	Target Latency (p99)	Notes
Single HyperCell read	< 5ms	< 20ms	Direct primary key lookup on dim_keys tuple
HyperRow read (all fields)	< 10ms	< 40ms	Batched read of all D ₂ keys for one D ₁ entity
DimSlice query (N=2, < 1K rows)	< 50ms	< 200ms	With PostgreSQL GIN/BTree index
DimFold aggregation (N=3, < 100K rows)	< 500ms	< 2s	With PostgreSQL; ClickHouse for larger
Graph traversal (depth 5, fan-out 10)	< 200ms	< 800ms	PostgreSQL recursive CTE or Neo4j depending on graph size

Operation	Target Latency (p50)	Target Latency (p99)	Notes
AS_OF time-travel query	< 100ms	< 500ms	EventLog replay for a single HyperRow
CRDT merge (concurrent edit)	< 1ms	< 10ms	In-memory merge; async persistence
AI attribute computation (Tier 2)	100ms – 30s	< 60s	Async; result written via WritebackService after computation
CrossGridLink delta sync	< 1s	< 5s	Kafka delta propagation + ShadowCell update

28.2 Scale Targets

Dimension	Target Scale	Notes
HyperRows per Hypercube	100M+	With PostgreSQL horizontal sharding by grid_id
HyperCells per Grid (total)	10B+	Universal Cell Store with sharded PostgreSQL clusters
Concurrent editors per Hypercube	1,000	CRDT buffer capacity per cube; beyond this: shard by entity range
Federation nodes per Grid	255	VectorClock slot limit
Events in hot EventLog	10,000 per entity	Flushed to S3 archive beyond cap; restorable on demand
CrossGridLink edges per identity	100,000+	KLNK graph; Neo4j backend for traversal at this scale
AI computation queue depth	10,000 pending	Kafka-backed queue; auto-scaling AI engine fleet

Part VIII — Extension, Plugin System & Domain Packs

29. The HypercubePlugin Interface

The HypercubePlugin trait is the primary extension point for the entire Apapo platform. Every domain-specific behavior that is not in the Hypergrid core — custom axis types, custom attribute types, domain-specific AI engines, custom render modes, and data connectors — is implemented as a HypercubePlugin. Plugins are loaded at Grid startup and registered with the PluginRegistry.

```
pub trait HypercubePlugin: Send + Sync {
    fn plugin_id(&self) -> &str;
    fn name(&self) -> &str;
    fn version(&self) -> &str;

    // Optional: register custom DimensionAxis types
    fn axis_type_definitions(&self) -> Vec<AxisTypeDefinition> { vec![] }

    // Optional: register custom attribute key definitions
    fn attribute_key_defs(&self) -> Vec<AttributeKeyDef> { vec![] }

    // Optional: register custom EdgeType patterns in the Hypergraph
    fn edge_type_definitions(&self) -> Vec<EdgeTypeDefinition> { vec![] }

    // Optional: provide a Tier-2 AI engine adapter
    fn ai_engine(&self) -> Option<Box<dyn AIEngineAdapter>> { None }

    // Optional: provide a custom rendering adapter
    fn render_adapter(&self) -> Option<Box<dyn RenderAdapter>> { None }

    // Optional: provide a data source connector
    fn data_connector(&self) -> Option<Box<dyn DataConnector>> { None }

    // Called on startup to register all Hypercubes this plugin owns
    fn bootstrap(&self, grid: &mut Grid) -> HypergridResult<()>;

    // Called for Tier-1 formula computation on registered attribute keys
    fn compute_attribute(
        &self,
        key: &AttributeKey,
        ctx: &ComputeContext<'_>,
    ) -> HypergridResult<TypedAttrValue>;
}
```

30. Domain Packs

A Domain Pack is a curated collection of HypercubePlugins bundled together to provide a complete domain-specific extension to the Qala Solution Factory. Domain Packs are the mechanism by which Qala supports governed solution production in highly regulated industries without requiring custom code in the platform core.

Domain Pack	Industry	Key Plugins Included	Regulatory Bodies
Pharma Domain Pack	Pharmaceutical	Batch formula versioning, GMP compliance, stability data, regulatory submission tracking, regulatory_risk AI	FDA, EMA, ICH
Financial Instrument Pack	Financial Services	KYC/AML module, risk model validation, regulatory capital computation, stress test orchestration	SEC, FCA, FINRA, Basel III
Medical Device Pack	Medical Device	Design history file, ISO 14971 risk management, V&V tracking, adverse event monitoring	FDA 21 CFR 820, ISO 13485
Software Pack (Default)	Software / Technology	Code metrics, test coverage, SAST/DAST integration, dependency audit, deployment validation	SOC 2, ISO 27001
Research Dataset Pack	Academia / Research	Provenance tracking, IRB approval, data anonymization, citation graph, reproducibility score	IRB, GDPR, NIH
Creative Works Pack	Creative / Media	IP ownership, license management, attribution chain, similarity detection, rights clearance	Copyright offices, CMOs

Part IX — Comparative Analysis

31. Hypergrid vs. Conventional Databases

31.1 Hypergrid vs. Relational Databases (PostgreSQL, MySQL)

Relational databases store entities in normalized tables with foreign key relationships. Schema changes require migrations. Distributed consistency requires additional infrastructure (Galera, Citus, etc.). There is no native CRDT support, no built-in event sourcing, no N-dimensional query model, and no AI attribute computation.

Hypergrid is built on top of PostgreSQL but adds the N-dimensional coordinate model, CRDT-per-attribute semantics, the append-only EventLog, the Hypergraph relationship layer, and the AI computation framework above the relational storage layer. For pure OLTP workloads with simple schemas, PostgreSQL alone is faster and simpler. For complex domain systems that need all of the above features, Hypergrid provides them as a coherent platform rather than requiring each team to build them individually.

31.2 Hypergrid vs. Graph Databases (Neo4j, Amazon Neptune)

Graph databases excel at relationship-centric queries: shortest path, neighborhood traversal, community detection. They are not designed for the rich property model (N attributes per cell), the N-dimensional slicing model, CRDT distributed consistency, or AI attribute computation. Hypergrid's Hypergraph layer provides graph traversal on top of a full entity-property store, giving it both the graph relationships and the rich attribute model that pure graph databases lack.

For very large graph workloads (KLNK at platform scale with >1M nodes), Hypergrid's v2.5 roadmap includes a Neo4j backend for the Hypergraph layer. The two systems are complementary, not competing.

31.3 Hypergrid vs. CRDT Stores (Automerge, Yjs, Electric SQL)

CRDT libraries like Automerge and Yjs are designed for real-time collaborative document editing (Google Docs-style, character-level edits). They have no query language, no graph layer, no N-dimensional model, no schema registry, and no AI attribute computation. Hypergrid uses CRDT semantics at the attribute key level (not character level) — appropriate for business domain data — and combines them with a full entity store.

31.4 Hypergrid vs. OLAP Databases (ClickHouse, DuckDB, BigQuery)

OLAP databases are read-optimized batch analytics engines. They excel at aggregation over billions of rows but are not designed for OLTP (individual entity reads/writes), CRDT semantics, event sourcing, graph relationships, or AI attribute computation. Hypergrid's DimFold/DimExpand operations mirror OLAP cube operations, but Hypergrid is designed for both operational and analytical use simultaneously (HTAP). For very large analytical workloads, Hypergrid's recommended architecture is to stream to ClickHouse as a secondary analytics backend.

31.5 What Hypergrid Actually Is

Hypergrid is a domain modeling substrate, not a database. It is a layer that sits above databases and provides a unified programming model for building domain-specific systems. It answers the question: 'What if every entity in your system had, by default, distributed collaborative editing, full version history, time-travel queries, typed graph relationships, multi-dimensional slicing, per-attribute AI computation, and a permission model — with no custom code required?' Its intellectual ancestors are spreadsheets (N-dimensional model), event sourcing (append-only EventLog), CRDTs (distributed merge semantics), property graphs (Hypergraph layer), OLAP cubes (DimFold/DimExpand), and functional reactive systems (push-based AI recomputation).

Part X — Open Items, Decisions & Roadmap

32. Open Technical Decisions

Decision	Options	Recommendation	Target Version
N-dim index at scale (N≥4)	PostgreSQL composite index Z-order curve ClickHouse Hybrid	Hybrid: PG for N≤4; ClickHouse for N>4 analytics cubes	v1.1
CRDT conflict surfacing UX	Silent (LWW always) Notify (user review on conflict) Block (require resolution)	Notify for semantic conflicts (status lattice); Silent for counters/sets	v1.0
Shadow cell write-back governance	Open (any editor) Governed (proposal required) Opt-in per attribute	Opt-in per attribute: write_back_attrs configurable per CrossGridLink	v1.0
Max N (dimensionality limit)	8 12 16 Unlimited	Hard limit 16; recommended max 6 for UI usability	v1.0 (finalize)
Formula engine sandboxing	WASM sandbox Process isolation Interpreted AST JVM sandbox	Interpreted AST with resource limits (no arbitrary code execution)	v1.0
Federation identity model	Grid-level trust Namespace-level trust Identity-level trust All three	All three tiers: Grid(full) > Namespace(partial) > Identity(minimal)	v1.5
AI engine default	OpenAI-compatible API Local LLM (Ollama) No default (platform-provided only)	Multiple: OpenAI-compatible + local LLM option + no-AI mode	v1.0
Time-travel granularity	Day Hour Minute Second EventLog entry	EventLog entry: AS_OF any timestamp resolves to last EventLog entry before that time	v1.0
Status CRDT Lattice	LWW (current v1) Custom lifecycle Lattice (v2)	LWW in v1 for simplicity; full Lattice in v2 for governance enforcement	v2.0
Global KLNK graph backend	PostgreSQL recursive CTE Neo4j Amazon Neptune TigerGraph	PostgreSQL recursive CTE for <1M nodes; Neo4j for platform-scale KLNK	v2.5

33. Priority Open Items

Item	Priority	System	Description	Blocks
Persistence Layer Pluggability	P0	All	PortfolioStore trait abstraction for pluggable backends (SQLite, PostgreSQL, CouchDB). Currently PG only.	Multi-node deployment; offline mode
Status CRDT Lattice	P0	All	Implement custom lifecycle lattice for status merges. Currently status ops are LWW only.	Multi-node deployment; federated Spaces
Space CRDT Sync	P0	Kogi/ume	Extend CRDT federation to Space-scoped components. Space members on different nodes see consistent state.	Organization Space multi-node
Cross-Partition SplitPolicy	P0	Kogi	SplitPolicy defined but not enforced at Substrate layer for cross-partition CrdtOperations.	Multi-identity deployments
ShadowRow Real-Time Delta Sync	P1	Kogi/Qala	Kafka consumer for shadow sync; column-delta push instead of full row refresh. Performance critical.	Cross-portfolio visibility at scale
EventLog Data Lake Flush	P1	All	10K event cap risks history loss. Plugin hook to flush to ClickHouse/S3 before eviction.	Long-lived component audit trail
WritebackService Permission Model	P1	All	Harden AI writer permission checks. Currently trust-based; needs cryptographic engine identity.	Production AI engine integration
Formula Engine v1	P1	Kogi/Qala	Full ColumnComputer expression language evaluator with all listed functions including RELATED() and engine functions.	Custom column power users
KLNK Consent Flow	P1	Kogi	Consent workflow for Collaborates/InvestedIn edge types (invitation, column visibility config).	Cross-portfolio collaboration
Real-Time Collaboration UI	P2	All	WebSocket-based live cursor indicators, cell lock indicators, presence badges in sheet header.	Shared Workspace experience

34. Version Roadmap

Version	Milestone	Key Deliverables
v0.5 (Alpha)	Core Substrate	HyperCell model · N=2 dense encoding · AttributeKeyRegistry · EventLog · PostgreSQL persistence · Basic REST API · LWW CRDT
v1.0 (Beta)	2D+ Platform	N=1–6 dimensions · Sparse + Hybrid encoding · HyperQL v1 · Formula language · View Engine (10 render modes) · Federation v1 · Spaces v1 · HG-NS v1 · Go SDK + TypeScript SDK · Kogi v3.0 + Ume v1.0 + Qala v1.0
v1.1	Full N-dim + Graph	All 16 dimension types · HG-GRAPH full implementation · CrossGridLink + Shadow Cell protocol · Link forest traversal · N-dim index optimization · ClickHouse analytics integration
v1.5	Intelligence + Identity	HG-AI pluggable engine · OpenAI-compatible adapter · NL-to-HyperQL · Anomaly detection · HG-ID multi-tenant identity · TenantPartition · VisibilityMask · CrossTenantMerge
v2.0	Full Platform	All render modes including N-DimExplorer · Governance plugins (cooperative, democracy, multisig) · All export formats (Parquet, Arrow, GraphQL) · Plugin marketplace · Web-based N-dim exploration UI · Status CRDT Lattice
v2.5	Scale + Federation	Neo4j graph backend · Cross-Grid AI (federated ML models) · ZOrder curve multi-dim index · 255-node federation · GDPR/CCPA automated compliance workflows · KLNK platform-scale graph
v3.0	Autonomous Intelligence	Autonomous AI agents operating on cubes with approval-first workflows · Predictive DimFold models · Cross-Grid MatchEngine · Semantic layer (AI-generated cube descriptions + ontology mapping) · On-chain identity anchoring (DID)
v3.5 (Kogi)	Web3 + Decentralization	On-chain identity anchoring · Decentralized portfolio storage (IPFS + CouchDB) · Smart contract cooperative distribution · On-chain equity/cap table · Cross-federation KLNK traversal · DAO governance integration

Part XI — Appendices

Appendix A: Complete Glossary

Term	Definition
Apapo	The complete integrated platform ecosystem: Hypergrid substrate + Kogi + Ume + Qala domain operating systems. The name for the whole.
Hypergrid	The N-Dimensional Distributed Spreadsheet System — the universal substrate. Provides all data infrastructure to domain systems.
Grid	Root container for all Hypercubes, dimensions, tenants, namespaces, spaces, and graph structures within one deployment.
Hypercube	An N-dimensional grid $H=(D_1, D_2, \dots, D_N)$. The generalization of a spreadsheet sheet. Named $\{\text{domain}\}.\{\text{entity_type}\}$ by convention.
DimensionAxis (D_i)	One axis of a Hypercube: EntityAxis (D_1), PropertyAxis (D_2), or any custom type (D_3 – D_N).
HyperCell	A data point at an N-dimensional coordinate, carrying an N-attribute map (AttributeMap).
HyperRow	All cells sharing the same D_1 key — the canonical entity in a Hypercube. The generalization of a spreadsheet row.
Universal Cell Store (UCS)	The physical storage layer holding all HyperCells across all Hypercubes across all domain systems. Keyed by (grid_id, cube_id, dim_keys_tuple).
AttributeMap	$\text{HashMap}<\text{AttributeKey}, \text{TypedAttrValue}>$ — the N-attribute payload of a HyperCell.
AttributeKey	A registered string key in the AttributeKeyRegistry. The 'column name' generalized to apply across all dimensions.
DimSlice	A predicate applied to one or more dimension axes, producing a sub-cube or projection.
DimFold	Collapsing an axis by aggregating all its key values into a summary. Generalization of GROUP BY.
DimExpand	Expanding a dimension's key set as separate columns. Generalization of PIVOT.
HypercubeView	A saved, shareable configuration of DimSlices, DimFolds, axis remappings, filters, sorts, and render mode.
Hypergraph	The graph layer: typed, directed edges between HyperCells, HyperRows, Cubes, Spaces, and Grids.
CrossGridLink	A HypergraphEdge crossing Grid boundaries. The inter-grid link network atom. Requires consent.
ShadowCell	A read-only reflection in Grid A of a linked entity from Grid B, created by a CrossGridLink.

Term	Definition
MirrorAttribute	A specific attribute from a ShadowCell synced into the host Grid as a read-only computed attribute.
LinkForest	The complete set of all link trees rooted at a given identity — their full cross-grid network.
Space	A named, governed, bounded operational context within a Grid. Groups cubes, members, and governance.
Workspace	Active working session within a Space. Personalized cube/view arrangement + session state.
NamespacePath	Hierarchical URI addressing any entity: hypergrid://{grid}/{space_type}/{slug}/...
SovereignTenant	The real-world entity (person, org, or agent) that owns a root set of Hypercubes and Spaces.
VisibilityMask	N-dimensional DimSlice predicates defining what is visible to each observer type.
VectorClock	HashMap<NodeId, u64>. Logical clock for causal ordering of all mutations across federation nodes.
CRDT	Conflict-free Replicated Data Type — per-attribute semantics: LWW OR-Set Counter Lattice.
EventLog	Append-only, immutable log of every mutation. Every cell attribute change is an EventLog entry.
CrdtLog	In-flight CRDT operation buffer: operations pending application and federation peer sync.
HyperQL	The N-dimensional query language: SELECT, DimSlice WHERE, FOLD, EXPAND, TRAVERSE GRAPH, AS_OF.
HypercubePlugin	The extension interface for all Hypergrid customizations: axis types, attr types, AI engines, render modes, connectors.
AIEngineAdapter	Plugin interface for connecting any AI/ML service to provide Tier-2 computed attributes.
ComputedAttribute	A derived attribute value: Tier-1 (synchronous formula) or Tier-2 (asynchronous AI/ML signal).
WritebackService	The gRPC service accepting AI-computed values and writing them into HyperCells with full audit trail.
AS_OF	Time-travel query operator: AS_OF timestamp returns the state of the cube at that historical moment.
Federation	CRDT synchronization between two or more Grid deployments. Delta sync via Kafka; VectorClock-based causal ordering.
Domain Pack	A bundled HypercubePlugin collection providing domain-specific governance, compliance, and AI for a regulated industry.
Kogi	Independent Worker Operating System. Built on Hypergrid. Primary entity: PortfolioComponent.

Term	Definition
KIMDSS	Kogi Interconnected Master Distributed Spreadsheet System — the complete Kogi platform described in this document.
KLNK	Kogi Link Network — the inter-portfolio graph system: forests, trees, ShadowRows, MirrorColumns.
Oba	The Kogi platform AI assistant — the spreadsheet's AI chief-of-staff. Reactive, Proactive, and Autonomous modes.
kogi-engine	The Scala 3 intelligence engine consuming all portfolio events and writing back computed signals via WritebackService.
Ume	Business Operating System. Built on Hypergrid. 42 organization modules as domain-specific Hypercubes.
Qala	Solution Factory Operating System. Built on Hypergrid. Primary entities: SDE, Solution, CCR, Release.
SDE	Solution Development Environment — the primary workspace in Qala for building and governing a solution.
CCR	Change Control Request — a formal governance record for a proposed change to a governed solution.

Appendix B: CRDT Selection Flowchart

Use this decision tree to select the correct CRDT semantics for any attribute key in any domain system:

```

Is the attribute a set of values (tags, members, dependencies)?
  YES → OR-Set
  NO ↓

Is the attribute a monotonically increasing counter (restarts, views, likes)?
  YES → GrowOnlyCounter
  NO ↓

Is the attribute a monotonically increasing max value (version numbers, sequence numbers)?
  YES → MaxRegister
  NO ↓

Is the attribute a lifecycle state that must follow a partial order?
  YES → Lattice (ComponentStatus, SdeStatus, SolutionLifecycleState, ModuleLifecycleState)
  NO ↓

Is the attribute human-authored content or a configuration value?
  YES → LastWriteWins (name, description, config, payload, JSON blobs)
  NO → Consult domain architects

```

Appendix C: Domain Pack Plugin Implementation Example

```
// A Qala Domain Pack is a HypercubePlugin + attribute registrations + edge types
pub struct PharmaDomainPack;

impl HypercubePlugin for PharmaDomainPack {
  fn plugin_id(&self) -> &str { "qala.domain_pack.pharma.v1" }
  fn name(&self) -> &str { "Pharmaceutical Formulation Domain Pack" }
  fn version(&self) -> &str { "1.0.0" }

  fn attribute_key_defs(&self) -> Vec<AttributeKeyDef> {
    vec![
      // Fields added to qala.solutions for pharma formulations
      lww_text("batch_formula_version", "Batch Formula Version"),
      lww_text("pharmacopoeial_grade", "Pharmacopoeial Grade"),
      lww_json("ingredient_specs", "Ingredient Specifications"),
      lww_json("regulatory_submissions", "Regulatory Submissions"),
      lww_text("gmp_compliance_status", "GMP Compliance Status"),
      lww_json("stability_data", "Stability Study Data"),
      // AI-computed regulatory risk
      AttributeKeyDef {
        key: "regulatory_risk_score".into(),
        attr_type: AttributeType::Ai,
        write_permission: PermissionTier::System,
        computation: Some(AttrComputation::AiEngine {
          plugin_id: PHARMA_REGULATORY_AI_ID,
          model_name: "regulatory_risk_v2".into(),
          parameters: HashMap::new(),
        }),
        ..lww_number("regulatory_risk_score", "Regulatory Risk Score")
      },
    ]
  }

  fn compute_attribute(
    &self,
    key: &AttributeKey,
    ctx: &ComputeContext<'_>,
  ) -> HypergridResult<TypedAttrValue> {
    if key == "regulatory_risk_score" {
      let ingredients = ctx.current_attrs.get("ingredient_specs");
      let submissions = ctx.current_attrs.get("regulatory_submissions");
      let score = self.regulatory_risk_engine.compute(ingredients,
submissions);
      return Ok(TypedAttrValue::AiSignal {
        value: Box::new(TypedAttrValue::Number(score.value)),
        confidence: score.confidence,
        model: "regulatory_risk_v2".into(),
        computed_at: Utc::now(),
      });
    }
    Ok(TypedAttrValue::Null)
  }
}
```

Appendix D: Key Design Decisions Summary

Decision	Chosen Approach	Rationale
Primary entity storage layout	$D_1 = \text{EntityId}$, $D_2 = \text{FieldName}$, <code>cell.attributes['value']</code>	Maximizes HyperCell reuse of built-in 'value' attribute; simplest codec pattern
Status/lifecycle CRDT	LWW in v1; Lattice in v2	LWW is safe and simple; Lattice adds governance enforcement at substrate level
AI writeback mechanism	<code>HypercubePlugin::compute_attribute + System PermissionTier</code>	Consistent with plugin architecture; prevents user overwrite of AI scores
Cross-system integration	<code>CrossGridLink + ShadowCell</code>	Consent-gated; attribute-level granularity; no full data copy
Federation sync	<code>CrdtLog</code> delta sync over Kafka	Reliable delivery; causal ordering; scales to 255 nodes
Schema evolution	Additive changes are CRDT-commutative	Zero-downtime schema updates; destructive changes require governance gate
Query language	HyperQL (custom N-dim) + NL bridge	Native N-dim query support; NL bridge for user-facing features
Event bus	Dual-write to domain <code>EventLog</code> + HG <code>EventLog</code>	Unified audit trail + domain-typed querying
Namespace scheme	<code>{domain}://{space}/{entity_type}/{id}/</code>	Domain prefixes prevent collision; URI-shaped for HTTP integration
Hypercube naming	<code>{domain}.{entity_type}</code>	Clear ownership; prevents naming collision across domain systems
Max dimensionality	N=16 hard limit, N=6 recommended	Beyond 6 dims, UI usability degrades significantly; 16 covers all edge cases
Cell store encoding	Hybrid (dense for $D_1 \times D_2$, sparse for D_3+)	Best performance for common N=2 case while supporting higher-dim cubes efficiently
AI computation tier	Tier-1 (sync formula) + Tier-2 (async AI)	Separates fast deterministic computations from slow non-deterministic AI signals
Permission model	Attribute-level <code>PermissionTier</code> + row-level <code>VisibilityMask</code>	Finest-grained access control without per-cell overhead

End of Document

Apapo Platform · Complete System Design Document · v1.0 · March 2026

Hypergrid Substrate · Kogi Independent Worker OS · Ume Business OS · Qala Solution Factory OS

Confidential — Internal Use Only